Kotlin

Kotlin

Entry Point:

```
fun main() {
    println("Hello world!")
}
```

Kotlin

Variables *

In Kotlin, a **variable** is a named storage location used to hold data. Variables are declared using two keywords.

1. val (Immutable Variable): Read-only variable (like final in Java). Its value cannot be changed after assignment.
2. var (Mutable Variable): Can be reassigned after declaration.

Sytax:

```
val variableName: DataType = value  // Immutable variable
var variableName: DataType = value  // Mutable variable
```

Kotlin

Rules for naming a variable, function and class *

1. Names can contain letters, digits, underscores, and dollar signs
2. Names should start with a letter
3. Names are case sensitive ("myVar" and "myvar" are different variables)
4. Names should start with a lowercase letter and it cannot contain whitespace (suggested)
5. Reserved words (like Kotlin keywords, such as var, val, String, Int, class, main, fun) cannot be used as names

Kotlin

Variables and Data types:

In Kotlin, you declare a variable starting with a keyword, **val** or **var**, followed by the name of the variable.

Data type : **Integer**

```
val a: Int = 10      // assign only once (not changeable : immutable)
var b: Int = 20      // modification is possible : mutable
print("value of a is $a and b is $b")
```

Data type : **Double**

```
var a: Double = 10.5
var b: Double = 20.7
print("value of a is $a and b is $b")
```

Kotlin

Variables and Data types:

In Kotlin, you declare a variable starting with a keyword, **val** or **var**, followed by the name of the variable.

Data type : **Char**

```
val a: Char = 'K'
var b: Char = 'A'
print("value of a is $a and b is $b")
```

Note

$ references a variable or expression in a string template.

Data type : **String**

```
var a: String = "Varanasi"
var b: String = "Computer Science"
print("value of a is $a and b is $b")
```

Kotlin

Nullable variables

In Kotlin, you can use null to indicate that there's no value associated with the variable.

To declare nullable variables in Kotlin, you need to add a ? operator to the end of the type.

```
var age: Int? = null
println(age)
```

```
var age: Int? = null
age = 20
println(age)
```

Kotlin

Mathematical Operators:

Operator	Description	Example (x=10,y=3)	Result
+	Addition	x + y	13
-	Subtraction	x - y	7
*	Multiplication	x * y	30
/	Division	x / y	3 (Integer division)
%	Modulo (Remainder)	x % y	1
+= , -= , *= , /= , %=	Augmented assignment	x += 10	x = x + 10
++, --	Increment, Decrement	--x , --y, ++x, ++y	?

Kotlin

Comparison Operators:

Operator	Meaning	Example (x=10,y=5)	Result
==	**Equal to**	x == y	false
!=	**Not equal to**	x != y	true
>	**Greater than**	x > y	true
<	**Less than**	x < y	false
>=	**Greater than or equal to**	x >= 10	true
<=	**Less than or equal to**	y <= 4	false

Kotlin

Conditions:

Kotlin gives you flexible tools to control your program's flow. Use if, when, and loops to define clear, expressive logic for your conditions.

If expression

To use if in Kotlin, add the condition to check within parentheses ( ) and the action to take if the result is true within curly braces { }. You can use else and else if for additional branches and checks.

```
fun main() {

    val number: Int = 2
    if(number % 2 == 0)
    {
        print("EVEN")
    }
    else
    {
        print("ODD")
    }
}
```

Kotlin

Conditions and Loops:

```
fun main() {

    val number: Int = 2
    if(number % 2 == 0)
    {
        print("EVEN")
    }
    else
    {
        print("ODD")
    }
}
```

```
    if(condition check)
    {
        -----
    }
    else if (condition check)
    {
        -----
    }
    else
    {
        -----
    }
```

Kotlin

Conditions and Loops:

If expression

Example Questions:

1. Write a program to check if a number is positive or negative.
2. Write an if expression that returns "Hot" if the temperature is above 30 and "Cold" otherwise.
3. Compare two integers and print "They are equal" only if they match.
4. Check if a student's marks are between 40 and 100 to determine if they passed.

Kotlin

Conditions:

when expression

when is a conditional expression that runs code based on multiple possible values or conditions. *when* evaluates its argument and compares the result against each branch in order until one branch condition is satisfied.

```
val userRole = "Editor"

when (userRole) {
"Viewer" -> print("User has read-only access")
"Editor" -> print("User can edit content")

else -> print("User role is not recognized")
```

```kotlin
fun main() {

    val ticketPriority: String = "X"

    when (ticketPriority)
    {
        "Low", "Medium" -> println("Standard response time")
        else -> println("High-priority handling")
    }

   var x:Int = 2
   when{
       x==1 -> print("ONE")
       x==2 -> print("TWO")
       x==3 -> print("THREE")
       else -> print("ABOVE THREE")

   }
}
```

```kotlin
fun main() {

    val ticketPriority: String = "X"

    when (ticketPriority)
    {
        "Low", "Medium" -> println("Standard response time")
        else -> println("High-priority handling")
    }

   var x:Int = 2
   when{
       x==1 -> print("ONE")
       x==2 -> print("TWO")
       x==3 -> print("THREE")
       else -> print("ABOVE THREE")

   }
}
```

```
High-priority handling
TWO
```

```
fun main() {

    var id = 3
    var pass = "password"

    when {
        id == 2 && pass == "password" -> print("Login success")
        else-> println("Access denied")
    }

}
```

```
fun main() {

    var id = 3
    var pass = "password"

    when {
        id == 2 && pass == "password" -> print("Login success")
        else-> println("Access denied")
    }

}
```

```
Access denied
```

```kotlin
fun main() {

    var id = 3
    var pass = "password"

    when {
        id == 2 && pass == "password" -> print("Login success")
        else-> println("Access denied")
    }

}
```

```
Access denied
```

```
fun main() {

    var id = 3
    var pass = "password"

    when {
        id == 2 && pass == "password" -> print("Login success")
        else-> println("Access denied")
    }

    if (id == 2 && pass == "password"){
        println("Login Success")
    }
    else{
        println("Access Denied")
    }
}
```

```
Access denied
Access Denied
```

Kotlin

Conditions:

For Loops

Use the for loop to iterate.

```
fun main() {
    for (i in 1..10){
        print(" $i ")
    }
}
```

```
1  2  3  4  5  6  7  8  9  10
```

Kotlin

Conditions:

For Loops

Use the for loop to iterate through a collection, array, or range:

```
fun main() {

    println("Closed-ended range:")
    for (i in 1..6) {
        print(i)
    }
    // Closed-ended range:
    // 123456

    println("\nOpen-ended range:")
    for (i in 1..<6) {
        print(i)
    }
    // Open-ended range:
    // 12345

    println("\nReverse order in steps of 2:")
    for (i in 6 downTo 0 step 2) {
        print(i)
    }
    // Reverse order in steps of 2:
    // 6420

}
```

Kotlin

Variables and Data types:

In Kotlin, you declare a variable starting with a keyword, **val** or **var**, followed by the name of the variable.

Arrays

```
val numbers = arrayOf(1, 2, 3, 4, 5)
println(numbers[0])
println(numbers.size)
```

Kotlin

Variables and Data types:

In Kotlin, you declare a variable starting with a keyword, **val** or **var**, followed by the name of the variable.

Arrays

```
val numbers = arrayOf(1, 2, 3, 4, 5)
for (elements in numbers){
        println(elements)
}
```

Kotlin

Variables and Data types:

In Kotlin, you declare a variable starting with a keyword, **val** or **var**, followed by the name of the variable.

List

```
val numbers = listOf(1, 2, 3) // Immutable (cannot modify)
val mutableNumbers = mutableListOf(1, 2, 3) // Mutable (can modify)
mutableNumbersList.add(4) // Adds element
mutableNumberList.remove(4) // removes value = 4
mutableNumberList.removeAt(0) // removes value at position 0 i.e. 1
```

Kotlin

Functions *

A **function** in Kotlin is a reusable block of code that performs a specific task. Kotlin functions improve code readability, reusability, and modularity.

name

parameters

return type

```
fun birthdayGreeting(name: String, age: Int): String {
    val nameGreeting = "Happy Birthday, $name!"
    val ageGreeting = "You are now $age years old!"
    return "$nameGreeting\n$ageGreeting"
}
```

body

return statement

Kotlin

Functions

A **function** in Kotlin is a reusable block of code that performs a specific task. Kotlin functions improve code readability, reusability, and modularity.

```
fun myname(s: String){
     println("my name is $s")
}
fun main(){
 myname("Shashvat")
}
```

```
fun add(a: Int, b: Int):
Int{
     return a + b
}
fun main(){
print(add(5,4))
}
```

Kotlin

Functions *

A **function** in Kotlin is a reusable block of code that performs a specific task. Kotlin functions improve code readability, reusability, and modularity.

```
fun myname(name: String, age: Int){
      println("my name is $name and age is $age")
}

fun main(){
      myname("Shashvat",25)
      myname("Anuradha",25)
      myname(25, "Anuradha")
      myname(age=25, name="Shashvat")
}
```

Kotlin

Functions (Default Arguments) *

A **function** in Kotlin is a reusable block of code that performs a specific task. Kotlin functions improve code readability, reusability, and modularity.

```
fun myname(name: String = "Shashvat"){
      println("my name is $name.")
}

fun main(){
      myname()
      myname("Amit")
}
```

Kotlin

Functions (Default Arguments) *

A **function** in Kotlin is a reusable block of code that performs a specific task. Kotlin functions improve code readability, reusability, and modularity.

```
fun myname(name: String = "Shashvat"){
      println("my name is $name.")
}

fun main(){
      myname()
      myname("Amit")
}
```

Exercise: Write a Kotlin program to find some of digits

Take an integer (e.g., 123) and return the sum of its individual digits (1+2+3 = 6).

Exercise: Write a Kotlin program to find some of digits

Take an integer (e.g., 123) and return the sum of its individual digits (1+2+3 = 6).

```kotlin
fun sumDigits(n: Int): Int {
    var number = n
    var sum = 0
    while (number != 0) {
        sum += number % 10
        number /= 10
    }
    return sum
}

fun main(){
    print(sumDigits(123))
}
```

```
6
```

Exercise: Write a Kotlin program to find factorial of given number

Take an integer n (e.g., 5) and return the product of all positive integers less than or equal to n.

Exercise: Write a Kotlin program to find factorial of given number

Take an integer n (e.g., 5) and return the product of all positive integers less than or equal to n.

```kotlin
fun factorial(n: Int): Long {
    var result: Long = 1
    for (i in 1..n) {
        result *= i
    }
    return result
}

fun main(){
    print(factorial(5))
}
```

```
120
```

Kotlin

Kotlin Class *

Kotlin offers concise syntax for declaring classes. To declare a class, use the **class** keyword followed by the class name.

Classes provide blueprints from which objects can be constructed. An object is an instance of a class that consists of data specific to that object. You can use objects or class instances interchangeably.

As an analogy, imagine that you build a house. A class is similar to an architect's design plan, also known as a blueprint. The blueprint isn't the house; it's the instruction for how to build the house. The house is the actual thing, or object, which is built based on the blueprint.

Kotlin

Kotlin Class *

Just like the house blueprint specifies multiple rooms and each room has its own design and purpose, each class has its own design and purpose. To know how to design your classes, you need to get familiar with *object-oriented programming (OOP)*, a framework that teaches you to enclose data, logic, and behavior in objects.

Kotlin

Kotlin Class *

OOP helps you simplify complex, real-world problems into smaller objects. There are four basic concepts of OOP, each of which you learn more about later in this codelab:

- **Encapsulation.** Wraps the related properties and methods that perform action on those properties in a class. For example, consider your mobile phone. It encapsulates a camera, display, memory cards, and several other hardware and software components. You don't have to worry about how components are wired internally.
- **Abstraction.** An extension to encapsulation. The idea is to hide the internal implementation logic as much as possible. For example, to take a photo with your mobile phone, all you need to do is open the camera app, point your phone to the scene that you want to capture, and click a button to capture the photo. You don't need to know how the camera app is built or how the camera hardware on your mobile phone actually works. In short, the internal mechanics of the camera app and how a mobile camera captures the photos are abstracted to let you perform the tasks that matter.
- **Inheritance.** Enables you to build a class upon the characteristics and behavior of other classes by establishing a parent-child relationship. For example, there are different manufacturers who produce a variety of mobile devices that run Android OS, but the UI for each of the devices is different. In other words, the manufacturers inherit the Android OS feature and build their customizations on top of it.
- **Polymorphism.** The word is an adaptation of the Greek root *poly-*, which means many, and *-morphism*, which means forms. Polymorphism is the ability to use different objects in a single, common way. For example, when you connect a Bluetooth speaker to your mobile phone, the phone only needs to know that there's a device that can play audio over Bluetooth. However, there are a variety of Bluetooth speakers that you can choose from and your phone doesn't need to know how to work with each of them specifically.

Kotlin

Kotlin Class *

A class consists of three major parts:

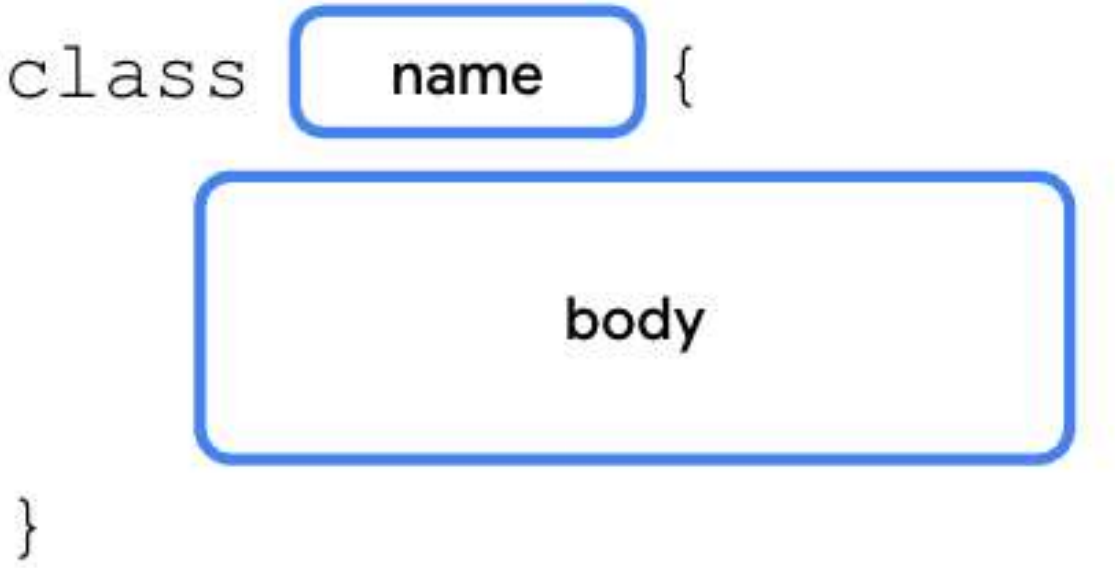
1. Properties. Variables that specify the attributes of the class's objects.

2. Methods. Functions that contain the class's behaviours and actions.

3. Constructors. A special member function that creates instances of the class throughout the program in which it's defined.

Kotlin

Kotlin Class *



Kotlin

Kotlin Class *

```
class MyClass {
 //empty body
}

fun main(){
}
```

Note: The class name is written in PascalCase, so each word begins with a capital letter and there are no spaces between the words. For example, in *S*martDevice, the first letter of each word is capitalized and there isn't a space between the words.

Kotlin

Kotlin Class *

Object: A **class** is like a blueprint for creating objects, just as a car company is used to make actual cars. An **object** is a real instance created from a class, holding data and performing actions. For example, if we have a Car class with properties like **brand** and **model**, we can create an object of that class to represent a real car. Objects help us organize and reuse code efficiently.

```
val name = ClassName()
```

```kotlin
class PhysicsCalculator(val version: String = "1.0") {

    // Speed: v = d / t
    fun calculateSpeed(distance: Double, time: Double): Double {
        return distance / time
    }

    // Distance: d = v * t
    fun calculateDistance(speed: Double, time: Double): Double {
        return speed * time
    }
}
```

```kotlin
class PhysicsCalculator(val version: String = "1.0") {

    // Speed: v = d / t
    fun calculateSpeed(distance: Double, time: Double): Double {
        return distance / time
    }

    // Distance: d = v * t
    fun calculateDistance(speed: Double, time: Double): Double {
        return speed * time
    }
}

fun main() {
    val obj = PhysicsCalculator()
    print(obj.calculateSpeed(120.0,2.0))

}
```